# Convenciones y Patrones de Diseño Base

Cuando la IA genera código sin restricciones, tiende a escribir "scripts secuenciales", es decir, código que se ejecuta de arriba a abajo sin estructura. Como Arquitecto, tu labor es imponer convenciones y patrones de diseño clásicos de la Ingeniería de Software que aseguren que el sistema sobreviva a largo plazo.

## 1. El Concepto Universal de Reutilización ("Componentización")

Aunque popularizado por React, el concepto de "Componente" es universal en la ingeniería de software y aplica tanto al Frontend como al Backend.

- **La Regla de Oro:** Un componente es una "Caja Negra" Agnóstica. El componente no debe saber *de dónde* vienen los datos ni *quién* los está usando. Solo recibe parámetros, los procesa y devuelve un resultado.
- **Ejemplo en Frontend (React):**
  - ❌ *Malo (Acoplado):* Un `<BotonPagar>` que por dentro hace el fetch a Stripe.
  - ✅ *Bueno (Agnóstico):* Un `<Boton />` genérico que recibe un `onClick` y un texto.
- **Ejemplo en Backend (Funciones):**
  - ❌ *Malo:* Una función de validación que intenta guardar en base de datos.
  - ✅ *Bueno:* Una función pura `validarEmail(email)` que solo retorna `true` o `false`.

## 2. Inyección de Dependencias

Nunca dejes que la IA inicialice librerías o conexiones complejas directamente dentro de las funciones de lógica de negocio.

- **El Problema:** Si la IA escribe el código de conexión a la BD dentro del módulo de facturación, no podrás probar ese módulo sin enviar datos reales a la BD.
- **La Solución:** Obliga a la IA a inyectar la dependencia.

```
// Mal: La función crea la conexión
function crearUsuario(datos) {
  const db = nueva ConexionDB("secret_key");
  db.guardar(datos);
}

// Bien (Inyección): La función recibe la conexión ya armada
function crearUsuario(datos, dbClient) {
  dbClient.guardar(datos);
}
```

## 3. Patrón Singleton (Esencial en Backend Serverless)

El Patrón Singleton asegura que una clase o conexión tenga una **única instancia** en todo el ciclo de vida de la aplicación.

**¿Por qué es crítico para Supabase / Edge Functions?**
En arquitecturas Serverless, la función se despierta y se duerme constantemente. Si instanciará un nuevo cliente por cada usuario, saturará el Connection Pool y tumbará el sistema.

> **Cómo exigirlo a la IA:**
>
> *"Para conectarte a Supabase, utiliza el Patrón Singleton en un archivo compartido (_shared/supabaseClient.ts). Garantiza que si el cliente ya fue instanciado, reutilice esa conexión en lugar de crear una nueva."*

## 4. Patrón Repository (Separación de Capas)

La regla de oro para bases de datos: **La interfaz visual (o el controlador) no habla con SQL.**

- **Implementación:** Debes exigirle a la IA que centralice todas las llamadas a la base de datos en un solo lugar llamado "Repositorio" o "Servicio".
- Si tienes que buscar usuarios, le pides a la IA que cree `UserRepository.ts` con funciones como `getUserById()`.
- **Beneficio:** Si mañana decides cambiar tu base de datos de PostgreSQL a MongoDB, solo tienes que reescribir el archivo `UserRepository.ts`. El resto de los archivos ni siquiera se enterarán del cambio, porque ellos solo llaman a la interfaz abstracta.